COMPANY SENTIMENT - MAPPA DEL PROGETTO

Data: 2026-06-21

1. Obiettivo Del Progetto

Il progetto serve a costruire una piccola applicazione che analizza il sentiment di testi aziendali o social.

In parole semplici:

- riceve un testo;
- usa un modello Hugging Face per classificarlo come positivo, neutro o negativo;
- salva le predizioni in un file CSV;
- legge un dataset gia' etichettato;
- confronta le predizioni del modello con le risposte corrette;
- salva le metriche nel tempo per poterle poi visualizzare in una dashboard.

2. Idea Generale

Ci sono due flussi principali.

Flusso A - Uso Normale Dell'Applicazione

client.py
manda un testo
|
v
POST /predict
|
v
model.py
usa il modello Hugging Face
|
v
schema.py
controlla che l'output abbia label e score
|
v
csvfiles_manager.py
salva su data/results.csv

Questo flusso risponde alla domanda:

Che sentiment ha questo testo?

Flusso B - Valutazione Del Modello

client_test.py
chiede un test sul dataset
|
v
POST /test?sample_size=N
|
v
newdata_manager.py
legge data/sentiment_datasets/train.csv
|
v
model.py
predice il sentiment di tanti testi
|
v
metrics.py
confronta predizioni e label vere
|
v
csvfiles_manager.py
salva su data/modelsummary.csv

Questo flusso risponde alla domanda:

Quanto sta andando bene il modello?

3. File Per File

src/API.py

E' il file principale dell'app FastAPI.

Contiene gli endpoint, cioe' le rotte che possono essere chiamate da browser, client Python o dashboard.

Endpoint attuali:

- 'GET /'
  - mostra una pagina HTML base;
  - serve solo come prova che l'app e' accesa.

- 'POST /predict'
  - riceve un singolo testo;

  - chiama il modello;
  - valida la predizione;
  - salva il risultato in ‘results.csv‘;
  - ritorna ‘label‘ e ‘score‘.

- ‘GET /results‘
  - legge ‘data/results.csv‘;
  - ritorna lo storico delle predizioni fatte con ‘/predict‘.

- ‘GET /summary‘
  - legge ‘data/results.csv‘;
  - ritorna un piccolo sommario: numero interazioni, conteggio sentiment, score medio.

- ‘POST /test?sample_size=N‘
  - prende N righe dal dataset;
  - fa predire il modello;
  - confronta predizioni e risposte corrette;
  - salva le metriche in ‘modelsummary.csv‘.

Nota importante:

‘/predict‘ serve per una frase singola.

‘/test‘ serve per valutare il modello su tante frasi del dataset.

src/model.py

Contiene il modello Hugging Face.

Il modello usato e’:

cardiffnlp/twitter-roberta-base-sentiment-latest

La funzione principale e’:

make_prediction(text, batch_size=16)

Puo’ ricevere:

- una stringa singola;
- una lista di stringhe.

Usa:

- ‘batch_size=16‘ per processare testi a blocchi;
- ‘truncation=True‘ per tagliare testi troppo lunghi;
- ‘max_length=512‘ come limite massimo supportato dal modello.

src/schema.py

Contiene i modelli Pydantic.

Servono a controllare che i dati abbiano la forma giusta.

Classi principali:

```
class Text(BaseModel):
    sentiment: str
```

Controlla l’input di ‘/predict‘.

```
class Prediction(BaseModel):
    label: str
    score: float
```

Controlla l’output del modello.

src/csvfiles_manager.py

Gestisce lettura e scrittura dei CSV.

File gestiti:

- ‘data/results.csv‘
  - contiene le predizioni fatte dall’app;
  - colonne principali: ‘time‘, ‘text‘, ‘label‘, ‘score‘.

- ‘data/modelsummary.csv‘
  - contiene le metriche del modello;
  - serve per confrontare test fatti in momenti diversi.

Funzioni principali:

- ‘read_file()‘
  - legge un CSV;
  - se il file manca o e’ vuoto, crea un DataFrame base.

- ‘prediction_saver(text, label, score)‘
  - crea una riga con timestamp, testo, label e score;
  - la aggiunge allo storico ‘results.csv‘.

- ‘df_summary()‘
  - crea un piccolo riassunto di ‘results.csv‘.

- ‘metrics_saver(metric_list, sample_size=None)‘
  - trasforma le metriche in una tabella;
  - aggiunge ‘time‘ e ‘sample_size‘;
  - aggiunge il nuovo report allo storico ‘modelsummary.csv‘.

src/newdata_manager.py

Legge il dataset pubblico.

Dataset attuale:

data/sentiment_datasets/train.csv

La classe principale e’:

Dataset("train.csv", "latin1")

Metodo principale:

return_text_target(sample_size=None)

Fa queste cose:

1. legge il CSV;
2. controlla che esistano le colonne ‘text‘ e ‘sentiment‘;
3. elimina righe con testo o label mancanti;
4. elimina testi vuoti;
5. se ‘sample_size‘ e’ impostato, prende solo le prime N righe valide;
6. ritorna:
   - ‘text‘: lista dei testi;
   - ‘target‘: lista delle label corrette.

src/metrics.py

Calcola le metriche del modello.

Usa:

classification_report

da scikit-learn.

Confronta:

- ‘target‘: label reali del dataset;
- ‘prediction‘: label predette dal modello.

Poi chiama:

csvfiles_manager.metrics_saver(...)

per salvare il report in ‘data/modelsummary.csv‘.

src/client.py

Client di test per ‘/predict‘.

Serve per mandare una singola frase all’API.

Esempio logico:

Mando: "normal company"
Ricevo: {"label": "neutral", "score": ...}

src/client_test.py

Client di test per ‘/test‘.

Serve per avviare una valutazione del modello sul dataset.

Esempio:

params = {"sample_size": 1000}
requests.post(url, params=params)

Significa:

Valuta il modello sulle prime 1000 righe valide del dataset.

4. File CSV

data/results.csv

Contiene lo storico delle predizioni singole fatte con ‘/predict‘.

Esempio:

time,text,label,score
2026-06-21 10:00:00,normal company,neutral,0.59

Serve a monitorare l’uso normale dell’app.

data/modelsummary.csv

Contiene lo storico delle metriche del modello.

Esempio:

time,sample_size,metric_class,precision,recall,f1_score,support
2026-06-21 10:00:00,1000,negative,0.68,0.80,0.73,280
2026-06-21 10:00:00,1000,neutral,0.77,0.51,0.61,420
2026-06-21 10:00:00,1000,positive,0.67,0.87,0.75,300

Serve a vedere se il modello migliora o peggiora nel tempo.

5. Come Avviare Il Progetto

Dal root del progetto:

uvicorn src.API:app --reload

Poi puoi aprire:

http://127.0.0.1:8000
http://127.0.0.1:8000/docs

6. Come Testare Le Parti

Per testare una predizione singola:

python src/client.py

Per testare il modello sul dataset:

python src/client_test.py

Se sei gia' dentro la cartella 'src':

python client.py
python client_test.py

7. Dove Siamo Arrivati

Il progetto ora ha:

- API FastAPI;
- modello Hugging Face;
- validazione base con Pydantic;
- salvataggio predizioni;
- lettura dataset pubblico;
- pulizia dati sporchi;
- calcolo metriche;
- storico metriche per confronti futuri.

Questa e' gia' una base forte per la traccia.

8. Cosa Manca Ancora

Prossimi passi principali:

1. Aggiungere una rotta per leggere 'modelsummary.csv'.
   - esempio: 'GET /model-metrics'

2. Creare un README serio.
   - obiettivo progetto;
   - modello usato;
   - endpoint;
   - come avviare;
   - come testare.

3. Creare 'requirements.txt'.
   - fastapi;
   - uvicorn;
   - pandas;
   - transformers;
   - scikit-learn;

- requests.

4. Aggiungere test semplici.
   - test lettura dataset;
   - test calcolo metriche;
   - test salvataggio CSV.

5. Piu' avanti: dashboard.
   - Grafana o Streamlit;
   - grafici su accuracy, f1_score, sentiment nel tempo.

6. Piu' avanti: deploy.
   - possibile Hugging Face Spaces.

9. Cose Da Ricordare

'GET':

chiedo dati al server

'POST':

chiedo al server di fare un'azione

'/predict':

una frase singola

'/test':

tante frasi dal dataset

'results.csv':

storico delle predizioni dell'app

'modelsummary.csv':

storico delle metriche del modello

10. Mini Glossario

Label:

risposta del modello: positive, neutral, negative

Score:

quanto il modello e’ sicuro della risposta

Target:

risposta corretta presente nel dataset

Prediction:

risposta generata dal modello

Accuracy:

percentuale di predizioni corrette

Precision:

quando il modello dice una classe, quanto spesso ha ragione

Recall:

quanti esempi reali di una classe riesce a trovare

F1-score:

media bilanciata tra precision e recall

Support:

quanti esempi ci sono per quella classe